

Linux Permissions Complete Command Guide

Basic Permissions | Special Permissions | Default/Formation Rules

Target Platform: RHEL 10 (RHCSA-aligned)

What's inside:

- How to read permission strings (`ls -l`) and octal notation
- `chmod`, `chown`, `chgrp` — symbolic & numeric usage with real examples
- Special permissions: SUID, SGID, Sticky Bit — command + purpose
- `umask` and default permission formation for files & directories
- Access Control Lists (ACL): `setfacl` / `getfacl` for fine-grained access
- Quick-reference command tables: command, usage, when to use it

1. Introduction to Linux Permissions

Every file and directory on a Linux system (RHEL 10 included) carries an **owner**, a **group**, and a set of **permission bits** that decide who may read, write, or execute it. Permissions exist in three layers, and this guide walks through all three, command by command:

- **Basic permissions** — read (r), write (w), execute (x) for owner, group, and others.
- **Special permissions** — SUID, SGID, and the Sticky Bit, which change how a file or directory behaves beyond the basic rwx model.
- **Default permission formation** — how the system decides the starting permissions of a newly created file/directory, controlled by **umask** and, optionally, **default ACL entries**.

Every command below includes its **usage** and **when to use it**, based on real RHCSA/RHEL 10 administration scenarios.

2. Reading Permission Strings

Running `ls -l` shows a 10-character string for each entry, for example:

```
-rwxr-xr-- 1 shafeeulla devops 4096 Aug 06 10:15 deploy.sh
```

This breaks down as:

Field	Meaning	Example value
Position 1	File type: - file, d directory, l symlink, c/b device	-
Positions 2-4	Owner permissions (read/write/execute)	rwx
Positions 5-7	Group permissions	r-x
Positions 8-10	Other (world) permissions	r--

Numeric (octal) equivalents: read = 4, write = 2, execute = 1. Add them per category to get a 3-digit mode, e.g. `rwxr-xr--` = 4+2+1, 4+0+1, 4+0+0 = **754**.

3. Basic Permissions

Basic permissions control three actions — read, write, execute — for three identity classes — owner (u), group (g), others (o). The commands below are the ones you use daily to view and change them.

3.1 chmod — Change Mode (permissions)

Usage: change the read/write/execute bits on a file or directory. Works in two notations: **symbolic** (u/g/o/a with +/-/=) and **numeric/octal** (0-7 per class).

```
# Symbolic notation
chmod u+x script.sh # add execute for owner
chmod g-w report.txt # remove write for group
chmod o=r notes.txt # set others to read-only exactly
chmod a+r shared.conf # add read for everyone (a = all)
chmod u+x,g+x deploy.sh # multiple classes in one command

# Numeric (octal) notation
chmod 754 deploy.sh # rwx r-x r--
chmod -R 750 /opt/appdata # recursive on a directory tree
```

When to use it: anytime a script won't execute ("Permission denied"), a shared file needs group write access, or you're locking down a config file so only the owner can edit it. -R is used when applying permissions recursively across a directory, e.g. web content or app data folders.

3.2 chown — Change Owner

Usage: change the user (and optionally group) that owns a file.

```
chown shafeeulla file.txt # change owner only
chown shafeeulla:devops file.txt # change owner and group together
chown :devops file.txt # change group only (chown form)
chown -R appuser:appgrp /var/www/app # recursive ownership change
```

When to use it: after copying files in as root that should belong to a service account (e.g. an app user for a systemd service), after restoring a backup, or when handing a project directory to another team/user.

3.3 chgrp — Change Group

Usage: change only the group that owns a file, without touching the owner.

```
chgrp devops project/ # change group ownership
chgrp -R devops /srv/shared # recursive group change
```

When to use it: setting up shared team directories where several users belong to the same group and need collective access, e.g. a /srv/shared folder for a DevOps team.

3.4 id / groups / whoami — Checking Identity

```
id shafeeulla # show UID, GID, and all supplementary groups
groups shafeeulla # list groups a user belongs to
whoami # show current effective username
```

When to use it: before changing ownership/permissions, confirm exactly which groups a user is in — this avoids granting access to the wrong group.

3.5 Quick Reference — Basic Permission Commands

Command	Usage	When to use it
chmod	Modifies rwx bits for owner/group/other, symbolic or octal.	Fixing script execute bit, restricting/opening file access.
chown	Changes file owner (and optionally group).	Handing files to a user/service account; after restoring backups.
chgrp	Changes only the group owner.	Setting up team-shared directories/files.
id / groups	Displays UID/GID and group membership.	Verifying identity before granting or auditing access.
stat	Shows detailed metadata incl. octal mode, UID, GID, timestamps.	Scripting/auditing when you need the exact octal permission value.

```
stat -c '%a %U %G %n' file.txt # e.g. '754 shafeeulla devops file.txt'
```

4. Special Permissions

Beyond `rxw`, Linux has three special permission bits that change *behavior*, not just access. Each has its own octal value used as a 4th (leading) digit in `chmod`.

4.1 SUID — Set User ID (octal 4)

What it does: when set on an **executable file**, the program runs with the privileges of the file's **owner**, not the user who launched it.

```
chmod u+s /usr/bin/myapp # symbolic: set SUID
chmod 4755 /usr/bin/myapp # numeric: 4 = SUID, then 755 = rwxr-xr-x
ls -l /usr/bin/myapp # owner 'x' shows as 's' -> -rwsr-xr-x
```

When to use it: classic example is `/usr/bin/passwd` — it needs to write to `/etc/shadow` (root-owned) even when run by a regular user. Use SUID only on trusted binaries; it is a common privilege-escalation risk if applied carelessly.

4.2 SGID — Set Group ID (octal 2)

What it does: on an **executable**, the process runs with the file's group privileges. On a **directory**, every new file/subdirectory created inside **inherits the directory's group** instead of the creating user's primary group.

```
chmod g+s /srv/shared_project # symbolic: SGID on a directory
chmod 2775 /srv/shared_project # numeric: 2 = SGID, 775 = rwxrwxr-x
ls -ld /srv/shared_project # group 'x' shows as 's' -> drwxrwsr-x
```

When to use it: the SGID-on-directory pattern is the standard way to build **team-shared folders** — every file a team member creates automatically belongs to the team group, so nobody needs to run `chgrp` manually afterward.

4.3 Sticky Bit (octal 1)

What it does: on a **directory**, it restricts deletion/renaming of files inside so that only the file's **owner** (or root) can remove it, even if others have write access to the directory.

```
chmod +t /srv/public_dropbox # symbolic: sticky bit
chmod 1777 /srv/public_dropbox # numeric: 1 = sticky, 777 = rwxrwxrwx
ls -ld /tmp # world-writable 'x' shows as 't' -> drwxrwxrwt
```

When to use it: `/tmp` is the textbook example — world-writable, but users can't delete each other's files. Use it on any shared, world/group-writable drop directory.

4.4 Combining and Finding Special Permissions

```
chmod 6755 file # SUID + SGID together (4+2)
chmod u+s,g+s,+t dir # symbolic combination of all three

# Locate files with special bits set (security auditing)
find / -perm -4000 -type f 2>/dev/null # all SUID files
find / -perm -2000 -type f 2>/dev/null # all SGID files
find / -perm -1000 -type d 2>/dev/null # all sticky-bit directories
```

When to use find: security audits and RHCSA exam tasks often ask you to locate every SUID/SGID binary on the system — this is the standard command for that.

4.5 Quick Reference — Special Permission Bits

Bit / Command	Symbol shown in ls -l	When to use it
SUID – chmod u+s / 4xxx	s (or S if no owner-x) in owner's x position	Executable must run with the file owner's privileges (e.g. passwd, ping).
SGID – chmod g+s / 2xxx	s (or S) in group's x position	On files: run with group privilege. On directories: force group inheritance for team folders.
Sticky – chmod +t / 1xxx	t (or T if no other-x) in other's x position	Shared/world-writable directories where users must not delete each other's files (e.g. /tmp).

5. Default Permission Formation (umask)

When a new file or directory is created, Linux does not start from 000 — it starts from a maximum baseline (666 for files, 777 for directories) and then **subtracts** the bits defined by **umask**. This is the "formation" of default permissions.

```
New file default : 666 (rw-rw-rw-) minus umask
New directory default: 777 (rwxrwxrwx) minus umask

Example with umask 022:
666 - 022 = 644 (rw-r--r--) -> typical new file
777 - 022 = 755 (rwxr-xr-x) -> typical new directory
```

Files never get an execute bit by default even if umask allows it — execute is only added explicitly or by the creating program (this is why new files are 644, not 666-minus-umask acting on an execute bit).

5.1 umask — View/Set the Default Mask

```
umask # show current mask, e.g. 0022
umask -S # show it in symbolic form, e.g. u=rwx,g=rx,o=rx
umask 027 # set mask for this shell session only
```

When to use it: quickly check or adjust what permissions new files will get in your current shell — useful before running a script or build process that creates many files, when you want tighter defaults (e.g. 027 hides files from "other").

5.2 Making umask Persistent

A umask set on the command line only lasts for that shell session. To make it permanent:

Scope	File to edit	When to use it
Single user	~/.bashrc or ~/.bash_profile	Apply a custom default only to your own login shell.
All users (system-wide)	/etc/bashrc or /etc/profile	Enforce an org-wide default, e.g. umask 027 for tighter security.
Specific service/daemon	systemd unit file (UMask= directive)	Control default permissions of files created by a service, independent of the shell umask.

```
# System-wide, e.g. in /etc/profile
umask 027

# systemd service unit
[Service]
UMask=0027
```

RHCSA Tip

In RHEL 10, the default umask is 022 for regular users and root under most login shells (via /etc/bashrc / /etc/profile logic based on UID). Know both the calculation (base – umask) and where to change it persistently — this is a common exam task.

6. Access Control Lists (ACL) — Beyond Owner/Group/Other

Basic permissions only support one owner and one group. When you need **multiple users or groups with different permission levels** on the same file, use POSIX ACLs. RHEL 10 filesystems (XFS, ext4) support ACLs by default.

6.1 getfacl — View ACL Entries

```
getfacl file.txt
```

When to use it: to inspect exactly who has access to a file beyond the standard owner/group/other, especially when troubleshooting "why can this user access this file?"

6.2 setfacl — Set ACL Entries

```
setfacl -m u:asha:rwX project.txt # give user 'asha' rwX
setfacl -m g:qa:rx project.txt # give group 'qa' read+execute
setfacl -x u:asha project.txt # remove asha's specific entry
setfacl -b project.txt # remove ALL extended ACL entries
```

When to use it: granting a specific extra user or group access without changing the file's actual owner/group — e.g. giving one auditor read access to a file that otherwise belongs to a different team.

6.3 Default ACLs on Directories — Formation for New Files

```
setfacl -m d:g:devops:rwX /srv/shared_project # default ACL: inherited by new files
getfacl /srv/shared_project # confirm 'default:' entries
```

When to use it: this is the ACL equivalent of umask/SGID for directories — any new file or subdirectory created inside automatically **inherits** this ACL entry, so permission "formation" stays consistent for a shared team folder without repeating setfacl every time.

6.4 Quick Reference — ACL Commands

Command	Usage	When to use it
getfacl	Displays all ACL entries (owner, group, extended entries, mask).	Auditing/troubleshooting access beyond standard rwX.
setfacl -m	Modifies/adds a specific user or group ACL entry.	Granting one extra user/group access without changing owner/group.
setfacl -x	Removes one specific ACL entry.	Revoking access previously granted to a single user/group.
setfacl -b	Removes all extended ACL entries at once.	Resetting a file back to plain owner/group/other permissions.
setfacl -d / -m d:	Sets a default ACL on a directory, inherited by new items.	Keeping consistent permission 'formation' for files created in a shared folder.

7. Consolidated Cheat Sheet

7.1 Octal Values at a Glance

Value	Basic permission	Special permission (leading digit)
4	read (r)	SUID
2	write (w)	SGID
1	execute (x)	Sticky Bit
0	no permission	none set

7.2 All Commands, One Table

Command	Category	One-line purpose
chmod	Basic	Change read/write/execute permissions.
chown	Basic	Change file/directory owner (and group).
chgrp	Basic	Change group ownership only.
id / groups	Basic	Check user identity and group membership.
stat	Basic	Show exact octal mode and metadata.
chmod u+s / 4xxx	Special	Set SUID — run as file owner.
chmod g+s / 2xxx	Special	Set SGID — run as group / inherit group on dirs.
chmod +t / 1xxx	Special	Set Sticky Bit — restrict deletion in shared dirs.
find -perm	Special	Locate SUID/SGID/sticky files for auditing.
umask	Default formation	View/set the mask subtracted from default perms.
getfacl	ACL	View extended ACL entries on a file/directory.
setfacl -m	ACL	Add/modify a specific user or group ACL entry.
setfacl -d	ACL	Set a default ACL so new files inherit it.

Practice Suggestion

On your ESXi homelab RHEL 9/10 control node: create a shared team directory, apply SGID + a group-write chmod, set umask 027 system-wide, then add a one-off ACL entry for a second user with setfacl. This single exercise touches every command in this guide.